Cryptography

Mingxun Zhou (mingxunz@ust.hk)

COMP2711 Discrete Mathematical Tools

1 Symmetric encryption

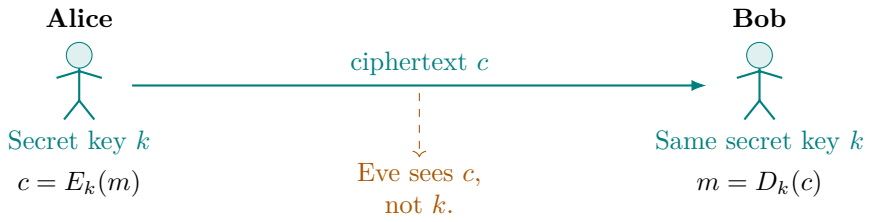
Alice wants to send Bob a private message over a public channel. Eve can observe everything sent through the channel. We study three tools: symmetric encryption, RSA public-key encryption, and Diffie–Hellman key exchange.

MUST UNDERSTAND

Definition (Symmetric encryption)**.** Alice and Bob share a secret key k. Alice encrypts a *plaintext* (original message) m to obtain a *ciphertext* c. Bob decrypts using the same key:

$$c = E_k(m), \qquad D_k(c) = m.$$

The encryption and decryption algorithms are public; only the key is secret. Correctness means $D_k(E_k(m)) = m$. Security asks whether Eve can learn the message without knowing k. Encryption may also use fresh randomness, which we omit from this notation.



1.1 Example: A substitution cipher

For a substitution cipher, the secret key is a letter-by-letter codebook: a table assigning each plaintext letter a ciphertext letter. Each ciphertext letter occurs exactly once in the table, so Bob can reverse the substitution.

Plaintext	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
Ciphertext	Q	W	E	R	T	Y	U	I	O	P	A	S	D	F	G	H	J	K	L	Z	X	C	V	B	N	M

For example, $M \mapsto D$, $E \mapsto T$, and $T \mapsto Z$. Applying the table letter by letter gives

$$\underbrace{\texttt{MEET AT NOON}}_{\text{plaintext}} \xrightarrow{\text{encrypt}} \underbrace{\texttt{DTTZ QZ FGGF}}_{\text{ciphertext}}.$$

Spaces are left unchanged in this example. Alice sends the ciphertext. Bob uses the same table in reverse, for example $D \mapsto M$ and $T \mapsto E$, to recover the plaintext. Eve observes the ciphertext but is not given the secret table.

Frequency analysis. Substitution preserves letter frequencies and repeated patterns. Eve can guess that a frequent ciphertext letter represents a common English letter such as E, then test the guess against word lengths and repeated letters. A large number of possible tables does not prevent this attack.

Historical example. During World War II, Nazi Germany used Enigma, a substitution-based encryption machine, for military communications. Its substitution changed as letters were typed. Alan Turing helped crack Enigma as part of a wider codebreaking effort, building on earlier Polish work.

1.2 Modern symmetric encryption: AES

The **Advanced Encryption Standard (AES)** was standardized by the US National Institute of Standards and Technology (NIST) in 2001 as FIPS 197. It is widely used in modern symmetric encryption. Unlike a substitution cipher, AES transforms blocks of bits rather than replacing individual letters. Both parties still need the same secret key. Practical encryption uses AES within a suitable mode of operation; we do not study its implementation here.

1.3 The key-distribution problem

A shared codebook or symmetric key must somehow reach both parties. Sending the key openly would reveal it to Eve as well. Two useful approaches are:

- **Public-key encryption:** Bob publishes an encryption key while keeping a related decryption key private. Alice can encrypt a chosen symmetric key for Bob, after which they can use that shared key for symmetric encryption.
- **Key exchange:** exchange public messages to establish a shared secret, then use it to derive a symmetric key.

Key exchange establishes a shared secret; encryption protects a chosen message.

2 RSA public-key encryption

2.1 Public and private keys

MUST UNDERSTAND

Definition (Public-key encryption)**.** Bob creates a public encryption key pk and a private decryption key sk. Anyone can encrypt to Bob using pk; Bob decrypts using sk. Correctness requires

$$D_{\text{sk}}(E_{\text{pk}}(m)) = m.$$

No pre-shared secret key is needed. Eve may know both the public key and the ciphertext.

2.2 The intuition behind RSA

The goal is *asymmetry*: everyone can encrypt with the public key, but only Bob can efficiently decrypt with the private key. In modern computers, messages are represented as bits, i.e., integers. For example,

$$\texttt{Hi} \longrightarrow \texttt{01001000 01101001} \longrightarrow 18537.$$

Addition, subtraction, and invertible multiplication are easy for Eve to undo when the constants and modulus are public. For example, if $c \equiv am + b \pmod{n}$ and $\gcd(a, n) = 1$, she can compute $m \equiv a^{-1}(c - b) \pmod{n}$.

Instead, try modular exponentiation: encrypt using $c = m^e \bmod n$ and decrypt using $c^d \bmod n$. We need suitable n, e, d such that $m^{ed} \equiv m \pmod{n}$. For $\gcd(m, n) = 1$, Euler's theorem gives

$$m^{ed} \equiv m^{\,ed \bmod \varphi(n)} \pmod{n}.$$

Here $\varphi(n)$ counts the integers from 1 to $n - 1$ that are coprime to n. Choosing $ed \equiv 1 \pmod{\varphi(n)}$ gives the required recovery for coprime messages; the proof below handles all messages for RSA's modulus.

If Eve knows $\varphi(n)$, she can also compute $d = e^{-1} \bmod \varphi(n)$ using extended Euclid. Bob therefore chooses distinct large primes p, q and publishes $n = pq$, while keeping the factors secret. Knowing them makes $\varphi(n) = \varphi(p)\varphi(q) = (p - 1)(q - 1)$ easy to compute. Multiplying the primes is easy, whereas factoring a suitably chosen large product is believed hard. This gives Bob an advantage: he can compute $\varphi(n)$ and d using the factors that Eve does not know.

2.3 The RSA algorithm

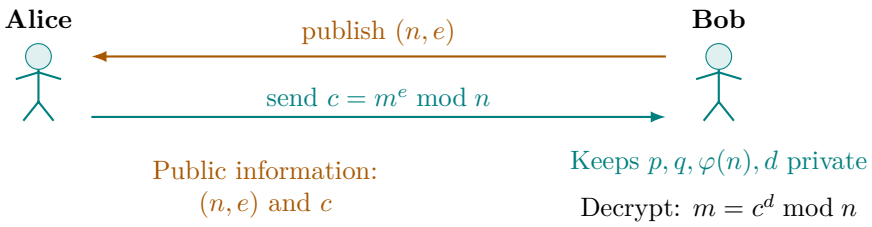
Key generation. Bob performs three steps:

1. Choose distinct large primes p, q. Set $n = pq$ and $\varphi(n) = (p - 1)(q - 1)$.
2. Choose $1 < e < \varphi(n)$ with $\gcd(e, \varphi(n)) = 1$.
3. Use extended Euclid to compute $d = e^{-1} \bmod \varphi(n)$.

Bob publishes $\text{pk} = (n, e)$. He keeps $p, q, \varphi(n), d$ private; $\text{sk} = d$ is the decryption key. The exponent e need not be chosen randomly; it must satisfy the coprimality condition.

Encryption and decryption. For an integer message $0 \le m < n$,

$$c = m^e \bmod n, \qquad \widehat{m} = c^d \bmod n.$$



Example. Take $p = 5$, $q = 11$. Then $n = 55$ and $\varphi(n) = 4 \times 10 = 40$. Choose $e = 3$. Euclid gives $40 = 13 \times 3 + 1$, so

$$1 = 40 - 13 \times 3, \qquad d \equiv -13 \equiv 27 \pmod{40}.$$

Indeed, $3 \times 27 = 81 = 1 + 2 \times 40$. Bob publishes $(55, 3)$ and keeps 27 private. For $m = 7$ the ciphertext is

$$c = 7^3 \bmod 55 = 343 \bmod 55 = 13.$$

Bob decrypts by computing $13^{27} \bmod 55 = 7$. These small numbers illustrate the algorithm; they are not secure parameters.

2.4 Why RSA decryption works

Theorem 2.1 (RSA correctness)**.** *With distinct primes p, q, $n = pq$, and positive integers e, d satisfying $ed \equiv 1 \pmod{(p-1)(q-1)}$, we have*

$$m^{ed} \equiv m \pmod{n}$$

for every integer m. Thus RSA decryption recovers every representative $0 \le m < n$.

Proof. Write $ed = 1 + t(p-1)(q-1)$ for an integer $t \ge 0$. First work modulo p. If $p \mid m$, then both m^{ed} and m are zero modulo p. If $p \nmid m$, Fermat's little theorem gives $m^{p-1} \equiv 1 \pmod{p}$, so

$$m^{ed} = m\big(m^{p-1}\big)^{t(q-1)} \equiv m \pmod{p}.$$

The same two cases modulo q give $m^{ed} \equiv m \pmod{q}$. As $\gcd(p, q) = 1$, the Chinese remainder theorem gives $m^{ed} \equiv m \pmod{pq}$. Finally $c^d \equiv (m^e)^d = m^{ed} \equiv m \pmod{n}$. Since decryption returns the representative in $\{0, \dots, n-1\}$, that representative is exactly m. □

When $\gcd(m, n) = 1$, Euler's theorem provides the shorter argument

$$m^{ed} = m\big(m^{\varphi(n)}\big)^t \equiv m \pmod{n}.$$

The prime-by-prime proof also covers non-coprime messages.

2.5 Why textbook RSA is not secure

Textbook RSA is deterministic: the same message always gives the same ciphertext. For example, if Alice always encodes *Yes* as $m = 1$ and *No* as $m = 0$, the ciphertexts are respectively 1 and 0, so Eve can read the answer directly. If Eve knows that m is one of two candidate messages, she can encrypt both with the public key and compare their ciphertexts. Actual RSA encryption schemes add randomized encoding; RSA-OAEP is specified in RFC 8017.

3 Diffie–Hellman key exchange

Alice and Bob each choose a private secret. They exchange public messages and compute a shared secret, from which they can derive a symmetric encryption key. Unlike public-key encryption, key exchange does not start with a message to encrypt.

3.1 Discrete logarithms

Repeated squaring efficiently computes $g^a \bmod p$. For example, $5^6 \bmod 23 = 8$. The reverse problem asks for the exponent.

Definition (Discrete logarithm problem)**.** Given a prime p and $g, A \in \{1, \ldots, p-1\}$, find $a \geq 0$ such that $g^a \equiv A \pmod{p}$, assuming a solution exists.

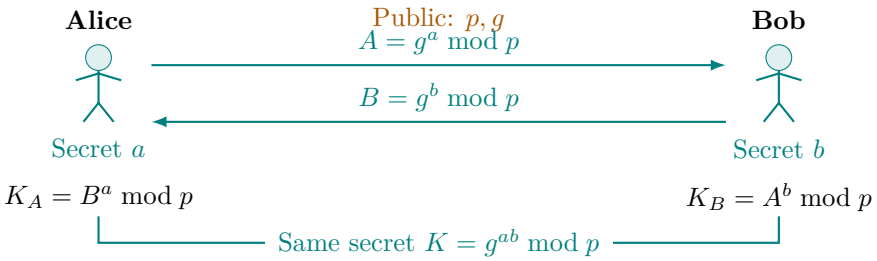
Any exponent satisfying the equation is acceptable. For suitably chosen large parameters, no efficient classical algorithm is known. The *discrete logarithm assumption* says that solving this problem is computationally hard.

3.2 The Diffie–Hellman protocol

MUST UNDERSTAND

Definition (Key exchange)**.** A key-exchange protocol lets Alice and Bob establish a shared secret using public communication, without revealing it to an eavesdropper.

In Diffie–Hellman, a prime p and a nonzero base g modulo p are public. Security requires suitably chosen large parameters. Alice and Bob choose their private exponents independently at random.



Both obtain the same value K, because

$$B^a \equiv (g^b)^a = g^{ab} = (g^a)^b \equiv A^b \pmod{p}.$$

Neither private exponent is transmitted. A practical protocol derives a symmetric key from the shared value rather than using its raw integer representation directly.

Example. Take $p = 23$, $g = 5$, $a = 6$, and $b = 15$.

$$\text{Alice sends } A = 5^6 \bmod 23 = 8 \qquad \text{Bob sends } B = 5^{15} \bmod 23 = 19,$$
$$\text{Alice computes } 19^6 \bmod 23 = 2 \qquad \text{Bob computes } 8^{15} \bmod 23 = 2.$$

Their shared value is $K = 2$. Eve sees $p = 23, g = 5, A = 8, B = 19$.

3.3 What an attacker must do

From g, g^a, g^b, Eve's target is g^{ab}, not necessarily either exponent. Recovering a by a discrete logarithm would suffice, since Eve could compute B^a. But this does not prove that every way of finding K must recover a. The assumption that computing the shared value is hard is called the computational Diffie–Hellman assumption in an appropriate group setting.

4 Security by reduction

A security reduction shows how an efficient attack on a cryptographic scheme would give an efficient algorithm for a computational problem:

$$\text{efficient attack} \implies \text{efficient algorithm for the problem.}$$

If we assume that the problem has no efficient algorithm, the reduction rules out such an attack. This connects cryptographic security to computational hardness.

The direction of the argument matters. Factoring n lets us break RSA by recovering $\varphi(n)$ and d; this alone does not prove that breaking RSA requires factoring. Similarly, solving discrete logarithms breaks DH, but key-recovery security uses the CDH assumption. A formal security claim must specify both the attack being ruled out and the computational assumption.

5 Summary

- Symmetric encryption uses a shared secret key.
- RSA uses a public encryption key and a private decryption key.
- Diffie–Hellman establishes shared secret material using public messages.

Modular exponentiation performs the operations, extended Euclid constructs RSA's inverse exponent, and Fermat and CRT establish RSA correctness.

6 Optional extensions

6.1 Man-in-the-middle attacks

Basic DH and RSA encryption do not, by themselves, authenticate the other party. In DH, Mallory can intercept the public values and establish a separate shared secret with each participant:

$$\text{Alice} \xleftrightarrow{K_1} \text{Mallory} \xleftrightarrow{K_2} \text{Bob}.$$

Alice and Bob think they are communicating with each other. Mallory can read, alter, and relay their messages. For RSA, Mallory can substitute her own public key for Bob's if Alice has no way to verify its owner.

Authentication prevents this impersonation. A public-key infrastructure (PKI) uses certificates to identify key owners, together with signatures to authenticate exchanges. Alternatives include authenticating with a pre-shared secret or checking public-key fingerprints over a trusted channel.

6.2 The CDH and DDH assumptions

Let $G = \{1, g, \ldots, g^{r-1}\}$ be a cyclic group of prime order r: these powers are distinct and $g^r = 1$. The group and its generator g are public. Choose a, b independently and uniformly from $\{0, \ldots, r-1\}$. In a Diffie–Hellman exchange, the public values are $A = g^a$ and $B = g^b$, and the shared secret is $K = g^{ab}$.

There are two different questions about what Eve can learn from A and B. Can she compute K? If she is given a candidate value, can she distinguish the true shared secret from an independent random group element?

Definition (Computational Diffie–Hellman (CDH))**.** Given g, $A = g^a$, and $B = g^b$, the CDH problem is to compute g^{ab}. The CDH assumption states that no efficient algorithm can solve this problem with non-negligible probability for random a, b.

Thus CDH concerns *recovering the shared secret*. It rules out an efficient method for computing K from the public exchange, but does not by itself say that K looks random to Eve.

Definition (Decisional Diffie–Hellman (DDH))**.** Choose z independently and uniformly from $\{0, \ldots, r-1\}$. The DDH problem is to distinguish the following two distributions:

$$\underbrace{(g^a, g^b, g^{ab})}_{\text{a real shared secret}} \qquad \text{and} \qquad \underbrace{(g^a, g^b, g^z)}_{\text{an independent random element}} .$$

The DDH assumption states that no efficient algorithm can distinguish these distributions with non-negligible advantage over random guessing.

Here the algorithm receives a triple (A, B, T); it is not given a, b, z. DDH says that even with A and B, Eve cannot reliably tell whether T is the real shared secret or a fresh random element.

Relationship to discrete logarithms. If Eve can compute a from $A = g^a$, she can solve CDH by computing B^a. If she can solve CDH, she can distinguish the DDH distributions by computing K and comparing it with T. Therefore DDH hardness is a stronger assumption than CDH hardness, which in turn implies discrete-logarithm hardness. Hard discrete logarithms alone do not guarantee DDH hardness.

These assumptions concern a chosen family of groups as their size grows. Here *efficient* means polynomial time in the bit length of the group parameters; *non-negligible* means at least an inverse-polynomial success probability or distinguishing advantage.

6.3 Digital signatures

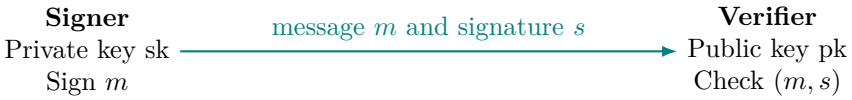
Encryption hides a message. A *digital signature* lets others check who signed a message and whether it has been changed.

Definition (Digital signature)**.** A signer generates a public verification key pk and a private signing key sk. To sign a message m, the signer computes

$$s = \mathrm{Sign}_{\mathrm{sk}}(m).$$

Anyone with pk can run $\mathrm{Verify}_{\mathrm{pk}}(m, s)$, which accepts a valid signature and rejects an invalid one.

Correctness requires verification to accept signatures produced by the signer. Security requires that, without the signing key, an attacker cannot efficiently forge a valid signature on a new message, even after seeing signatures on other messages. To identify the signer, the verifier must also know whose public key is being used, for example through a certificate.



RSA signatures. The signer applies the private exponent to an encoded message digest h, a representation derived from a hash of the message: $s = h^d \bmod n$. Verification uses the public exponent to recover $s^e \bmod n$ and checks the encoding against the message. A complete scheme such as RSA-PSS specifies the hashing, encoding, and verification rules; simply raising the raw message to d is not a secure signature scheme.

Other signature schemes. ECDSA (Elliptic Curve Digital Signature Algorithm) and EdDSA (Edwards-curve Digital Signature Algorithm) use elliptic-curve arithmetic rather than RSA's arithmetic modulo pq. They have the same basic interface: sign with a private key and verify with a public key. Their mathematical constructions are beyond this lecture.

7 Exercises

1. **Key roles.** Bob wants to receive a private message. Which key does Alice use in symmetric encryption? Which key does she use in public-key encryption? Which keys must Bob know?
2. **Diffie–Hellman.** Let $p = 23$, $g = 5$, $a = 8$, $b = 5$. Find both public values and the shared value. State what Eve observes.
3. **RSA.** Let $p = 5$, $q = 11$, $e = 3$ as in the worked example. Encrypt and decrypt $m = 5$. Does Euler's theorem apply directly to this m, n?
4. **Challenge: recover the private key.** Suppose you know that $n = pq$ for distinct primes and someone reveals $\varphi(n)$. How can you recover p, q? Try $n = 55$, $\varphi(n) = 40$.
5. **Deterministic encryption.** Alice uses textbook RSA to send "Yes" as $m = 1$ and "No" as $m = 0$. What does Eve learn? More generally, how can Eve test whether a ciphertext encrypts one of two known messages?
6. **Your turn: crack a substitution cipher.** Recover the original message in the following file. Use letter frequencies and repeated patterns.

   https://wuwuz.github.io/COMP2711/Materials/cipher.txt

Solutions to Exercises 1–5

1. Symmetric encryption uses the shared secret k for both operations. Public-key encryption uses Bob's public key for Alice's encryption and Bob's private key for his decryption.
2. Alice sends $A = 5^8 \bmod 23 = 16$; Bob sends $B = 5^5 \bmod 23 = 20$. Both compute $K = 20^8 \bmod 23 = 16^5 \bmod 23 = 6$. Eve sees $p = 23, g = 5, A = 16, B = 20$.
3. $n = 55$, $\varphi(n) = 40$, and $d = 27$. Encryption gives $c = 5^3 \bmod 55 = 15$; decryption gives $15^{27} \bmod 55 = 5$. Euler's theorem does not apply directly because $\gcd(5, 55) = 5$. RSA correctness still follows by working modulo 5 and 11, then applying CRT.
4. Since $\varphi(n) = pq - p - q + 1$, we know $p + q = n + 1 - \varphi(n)$. The two roots of $X^2 - (n + 1 - \varphi(n))X + n = 0$ are p and q. For the example, $p + q = 16$, and $X^2 - 16X + 55 = (X - 5)(X - 11)$. Once the factors are known, compute $d = e^{-1} \bmod \varphi(n)$.
5. The ciphertext is 1 for "Yes" and 0 for "No", so Eve reads the answer directly. For two known candidate messages, Eve encrypts each using the public key and compares the results with the observed ciphertext.

References

Rosen, *Discrete Mathematics and Its Applications*, 8th ed., Section 4.6; prior lecture results from Sections 4.1–4.4. For the optional DDH distinction, see D. Boneh, *The Decision Diffie–Hellman Problem*. For RSA encoding specifications, see RFC 8017. For digital signature standards, see NIST FIPS 186-5. For AES, see NIST FIPS 197. For Enigma and Turing, see The Alan Turing Institute. The codebook and all diagrams here are original teaching illustrations.